

Date: 5th April 2026



ANRF AISEHack 2026

Finale presentation

Theme : Flood prediction

Itikela Bhaskar, CVIT
Vijay Aravynthan, SERC

Team Megalodon @ IIIT-H

Presenting our solution

01

5-model Heterogeneous ensemble
with ConvNeXt and U-Net

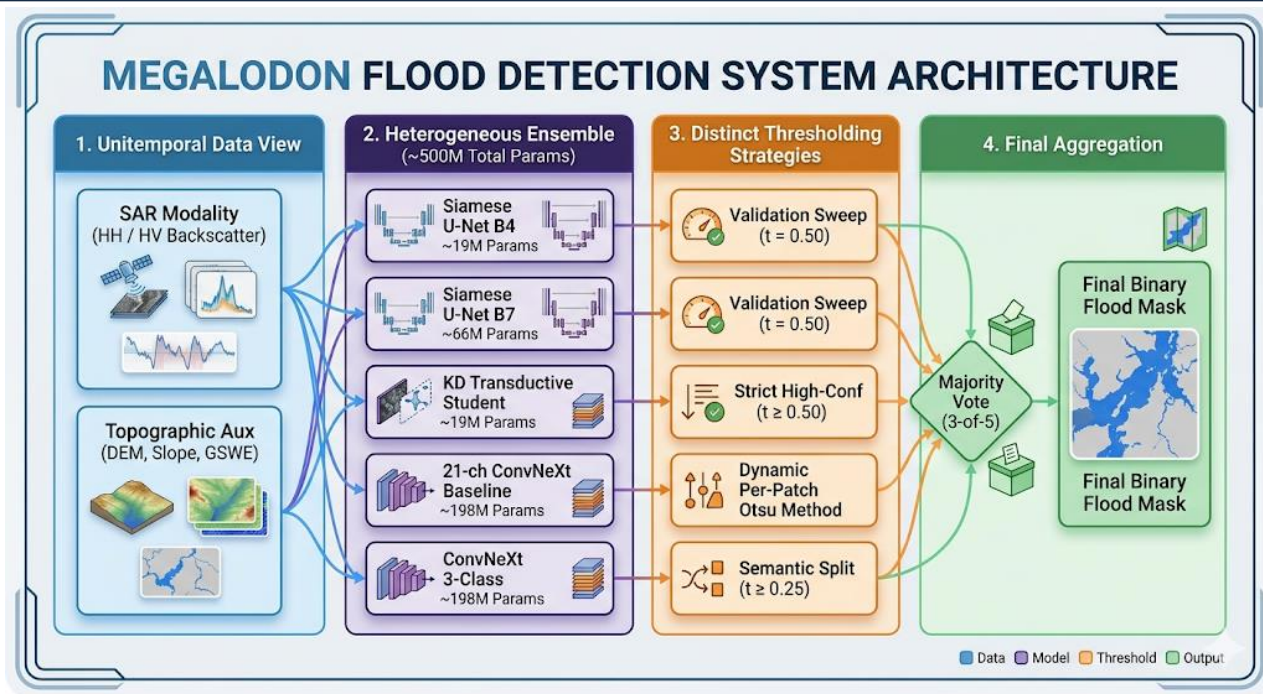
02

Trained mainly on SAR, DEM
and GSWE

03

3rd position in the leaderboard +
Easy to use notebook + basic
website

Model and techniques used



Strengths:

- When parallelized; the **ensemble model is faster than a colossal single-model (like the 300M parameter Prithvi)** because it leverages efficient CNN architectures which can be trained parallel.

Weaknesses:

- High VRAM memory footprint during training
- Tuning probability thresholds for an overlapping 5-model ensemble is brittle and demands highly careful held-out validation.

What is working?

S1

Architecture, Training strategy, Data strategy, and Hyperparameters

Architecture:

We unified **highly-efficient binary Siamese networks** with a **heavyweight 3-class ConvNeXt backbone** into a 5-model heterogeneous ensemble to maximize architectural diversity and error decorrelation.

Data Strategy:

We engineered a dense 21-channel tensor. We mathematically confined the **raw SAR backscatter (HH/HV)** by **stacking it alongside critical static topographies (DEM elevations, Slopes, and GSWE permanent water masks)** and other augmentations like local variance.

Training Strategy:

The absolute core driver of our convergence was deploying **Edge-Aware Loss functions mixed with heavily Weighted BCE** to force the network to handle extreme class imbalances. We also finalized our checkpoints gracefully using **SWA (Stochastic Weight Averaging)**.

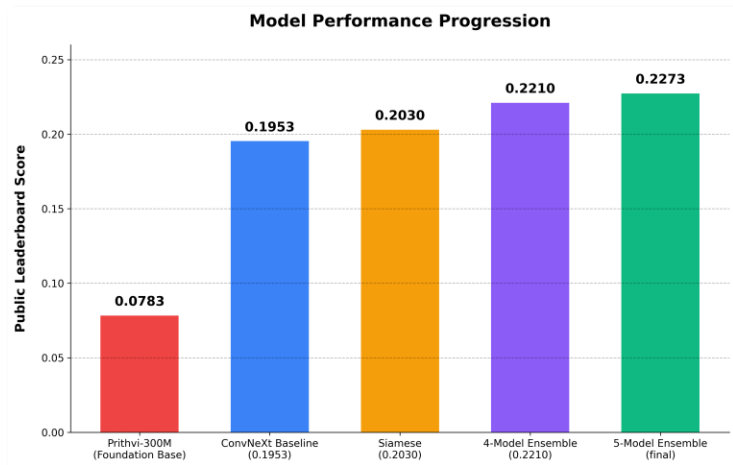
Hyperparameters:

We broke away from generic setups by deploying **Dynamic Per-Patch Otsu thresholding** rather than arbitrary global floors. We also set a **severe learning rate decay (down to 2e-6)** during the final fine-tuning epochs to lock in the optimal weights. Additionally, We implemented a **min-size threshold** to pervert spurious flood predictions.

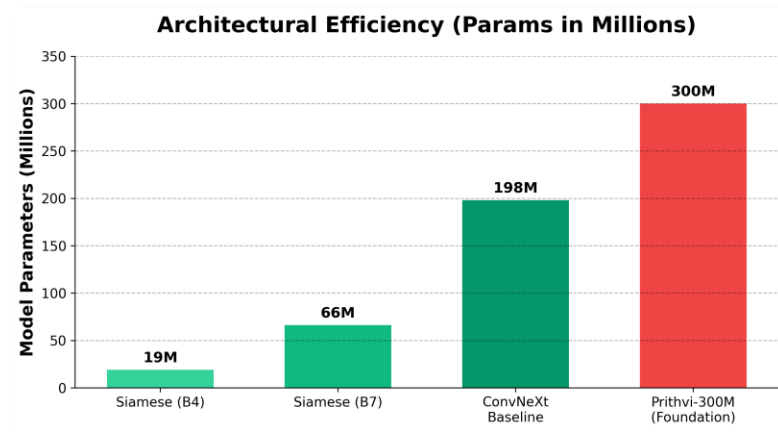
How is it working? – 1/2

S2

Quantitative evals



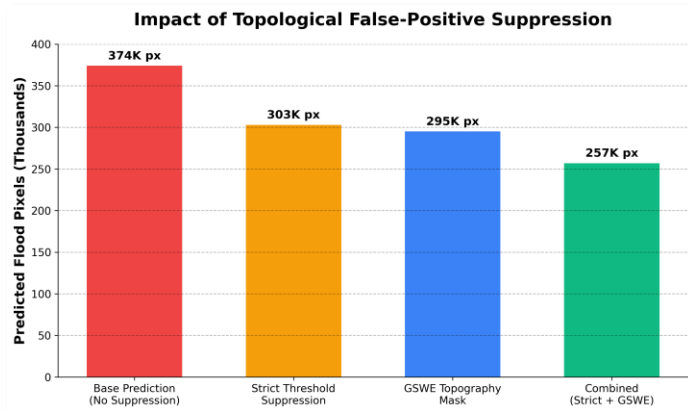
Quantitative Leap: Our architecture propelled our standalone baseline (0.1953 LB) up to a final cohesive Public LB score of 0.2273 via a strict 3-of-5 majority vote.



Efficiency & Hardware: By rejecting 300M+ parameter foundation models, we kept our models lean (ranging from 19M to 198M params).

How is it working? – 2/2

Post processing: We can see how post processing of the pixels helps reduce the number of misclassified flood pixels.
Otsu also helps here.



S4

Shift phase 1 to phase 2

Phase 1 was just binary semantic segmentation with clear divisions between land and water.

Phase 2 forced us to **use heavily engineered "anomaly detection."** using external data sources like **DEM** and **GSWE**.

We couldn't rely on raw deep learning to figure it out organically—we had to manually construct the topological physics for the network.

Why is it working?

S3

Reasoning and analysis

Isolating SAR Physics: Standing water reflects radar like a mirror, creating dark voids. By explicitly feeding the network mathematical shortcuts (like HH/HV log ratios and local spatial variances), the CNNs pinpointed this specific attenuation perfectly.

Topographic Filtering: SAR notoriously hallucinates floods inside mountain shadows. By embedding digital elevation (DEM) deeply into the modeling channels, the network learned the fundamental rule that water biologically cannot pool on steep inclines.

Error Decorrelation: Our models are fundamentally different from one another (binary matching vs. 3-class semantic mapping). Because they make completely different mistakes, the 3-of-5 voting filter ruthlessly scrubs out individual edge-case errors, leaving only the truest boundaries behind





Results

These are our **final results**

In the crucial metric of “insert” our model performs exceptionally well.

For inference time per sample on a laptop Nvidia 4070, it takes 0.48 seconds. The same inference on a desktop Nvidia 4090 takes 0.17 seconds.

Metric	Value
LB Score	0.2273
Mean IoU	0.4209
Flood IoU	0.2239
Overall accuracy	0.6558
Time for Inference	0.17 s

Details of Experimentation

Bonafilia_Sen1Floods11 which is landmark paper for flood detection (Sen1Floods11) uses CNN architecture (U-net along with SAR+DEM) data to achieve results. Attentive **Dual Stream Siamese U-net** outlined in this 2022 paper

Sen1Floods11: a georeferenced dataset to train and test deep learning flood algorithms for Sentinel-1

Derrick Bonafilia¹
Cloud to Street¹
www.cloudtostreet.info

Beth Tellman^{1,2}
Earth Institute, Columbia University²
beth@cloudtostreet.info

Tyler Anderson¹
tyler@cloudtostreet.info

Erica Issenberg¹
erica@cloudtostreet.info

ATTENTIVE DUAL STREAM SIAMESE U-NET FOR FLOOD DETECTION ON MULTI-TEMPORAL SENTINEL-1 DATA

Ritu Yadav,, Andrea Nascetti, Yifang Ban

Division of Geoinformatics, KTH Royal Institute of Technology, Sweden

Over the course of the hackathon, we had 4 key observations:

- 1) The models tends to focus on **the SAR data** as the key metric (identified with SHAP)
- 2) External data such as **HAND** and **GSWE** boosted performance significantly
- 3) Very **small number of flood pixels** in the training data; discrepancy in illumination of the test and train
- 4) Global thresholding provided bad results



Why Prithvi itself fails and the “MAJOR” issue we faced

We tried three of the state-of-the-art Prithvi EO-2 models

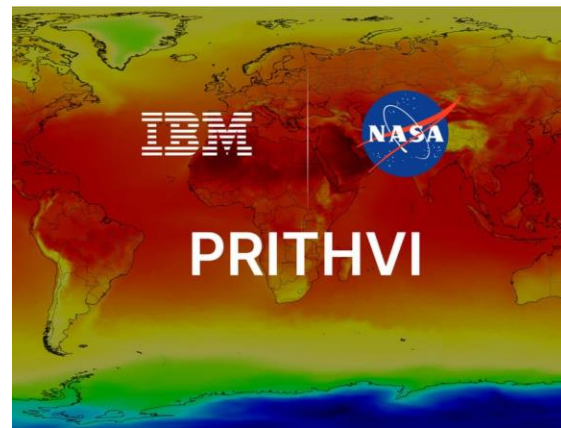
- [Prithvi-EO-2.0-300M-TL-Sen1Floods11](#) (Pre fine tuned)
- [Prithvi-EO-2.0-300M-TL](#)
- [Prithvi-EO-2.0-600M-TL](#) (Very large model)

These 3 models while excellent for optical tasks, really struggled with the flood prediction task.

They were only able to get a LB score of around 0.13-0.15

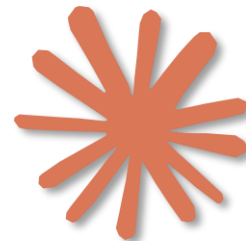
The 2 main theories we had were:

- 1) The model was unable to use very important data like DEM and GSWE due to being a ViT not being trained with the same.
- 2) The model didn't have sufficient data to adapt to the flood prediction domain.

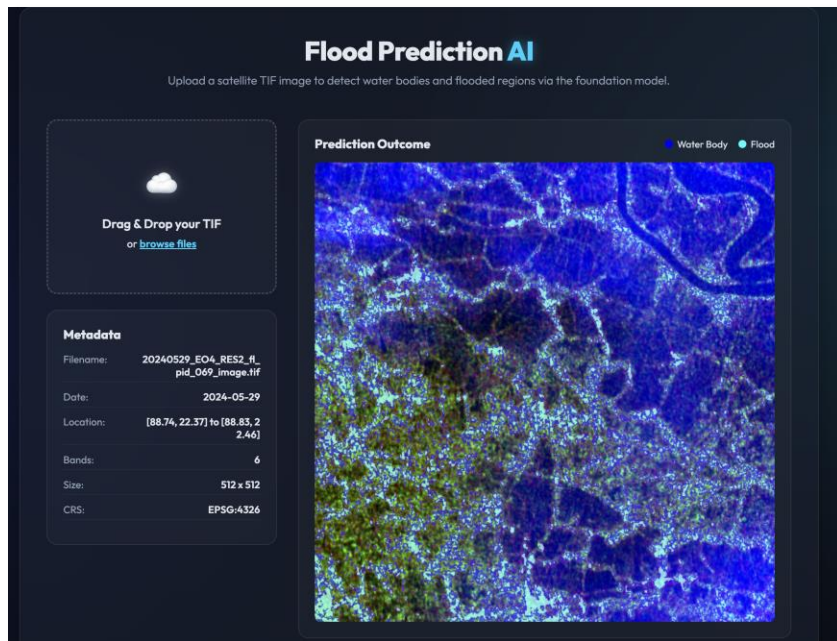


Details of Experimentation

- The model train was done using a **Nvidia 4090** depending on the model architecture – training each epoch took a **0.5 - 3 mins**
 - As we had limits on its usage, we prioritized ensembles and post training finetuning. (Which didn't really help that much)
 - Training took around **8GB of VRAM**
-
- While we used agentic AI for domain knowledge and speeding up the coding extensively, we didn't use any specific prompting techniques.
 - We maintained context throughout the large number of experiments by using a **common scratch pad which is updated with the experiment results and diagnosis**



Future plans



- Though we didn't have time to fully kit it out with all the features we wanted.
- We built a basic website for our model.
- We will deploy it on Hugging Face Spaces.

Deep learning-based segmentation of multi-temporal satellite imagery for flood detection

Vita Kashtan^{*,†} and Volodymyr Hnatushenko[†]

Dnipro University of Technology, Dmytra Yavornytskoho Ave 19, Dnipro, 49005, Ukraine

Abstract

A deep learning-based pixel-based flood zone segmentation approach is proposed using multi-temporal satellite images and topographic and hydrological information. It is proposed to combine heterogeneous data (satellite images before and after the flood, digital elevation model, and hydrographic characteristics) into a single input tensor, allowing the neural network to consider the area's spatial and temporal dynamics and morphometric features. The architecture of the model ensures the preservation of the spatial detail of the flooded area through skip-connection mechanisms, which contributes to the correct identification of flood boundaries. Comparative analysis with FCNN, DeepLabv3, and BASNet confirmed the superiority of the proposed approach (F1-score 82%, Dice 82% for the category 'flooded areas'), which indicates its effectiveness for accurately detecting flooded areas.

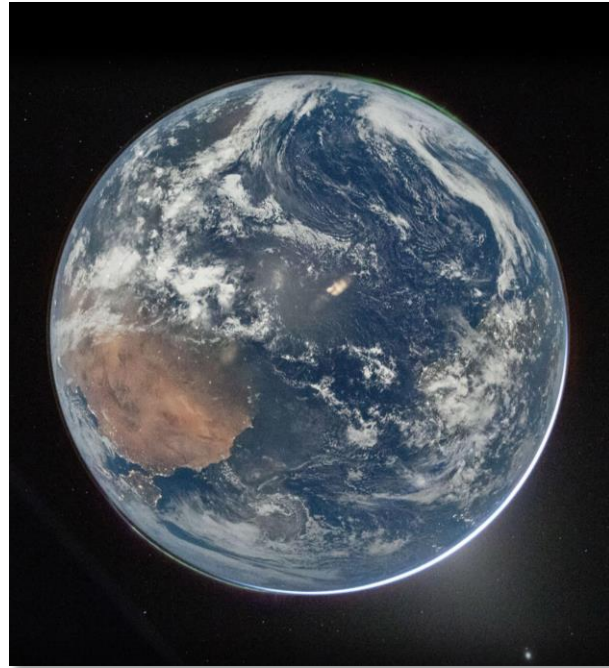
Keywords

deep learning, pixel segmentation, flooding, multi-temporal satellite imagery, neural network, classification

- We still feel that our model has massive scope for improvement. (LB score of 0.22 out of 1)
- We hope to continue working on this over the summer and explore more datasets.
- Like this paper

Thank you

Feel free to ask us any questions



Our planet 🌍 captured by the current Artemis 2 mission 🚀